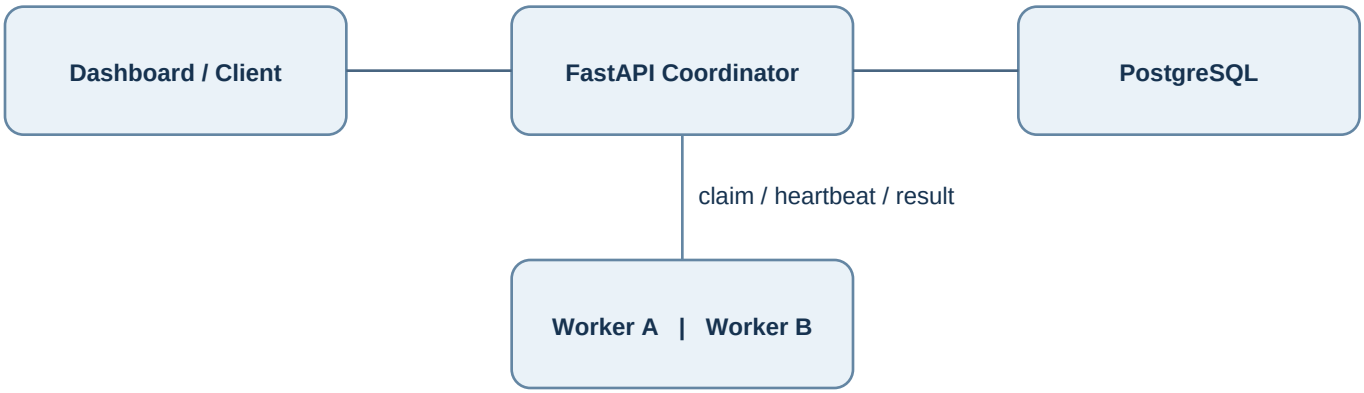
MeshCompute

How the distributed task system works



A practical guide to submitting jobs, coordinating workers, recovering failures, and running the local project. The database is the durable source of truth.

1. One job from request to result

POST /api/v1/jobs validates a registered handler and structured input.

The coordinator saves the job and its task rows in one database transaction.

It returns HTTP 202 with a job ID; workers claim tasks asynchronously.

Workers execute registered handlers, heartbeat, then report results.

When all tasks reach a terminal state, partial results are aggregated.

Concurrency and recovery

PostgreSQL row locks with SKIP LOCKED prevent workers claiming the same row.

Each claim increments an attempt number and sets a lease expiration.

Heartbeats extend leases. The reaper retries work after a worker disappears.

Attempt fencing prevents stale workers from overwriting a newer attempt.

Delivery is at least once, so handlers should be deterministic/idempotent.

Code boundaries

domain: state rules and framework-independent data.

services: submission, scheduling, leases, completion and aggregation.

repositories/db: SQL persistence and transaction boundaries.

coordinator/api: REST and WebSocket boundaries.

worker: API client, heartbeat, process isolation, registered handlers.

dashboard: read-mostly jobs, workers, metrics, and detail view.

2. Run and explore

Requirements: Docker Desktop engine running; Python 3.12+ and Node.js 20+ for dev.

1. Copy .env.example to .env
2. docker compose up --build
3. Dashboard: http://localhost:3000
4. API documentation: http://localhost:8000/docs
5. Submit sample work: python scripts/run_demo.py
6. Stop: docker compose down (database volume is preserved)

Checks: make install, make lint, make typecheck, make test, make check.

PostgreSQL integration/failure checks require a functioning Docker engine.

Design choices and limits

The built-in handlers are explicitly registered; callers cannot submit code.

The demo image resize path accepts uncompressed 24-bit BMP only.

WebSocket messages are best-effort; the dashboard refreshes from the API.

This reference project does not claim exactly-once side effects.

No performance numbers are claimed until the benchmark is run and recorded.

Suggested reading: README.md, docs/architecture.md, docs/operations.md, then

follow coordinator API -> service -> repository -> worker -> handler.